IQRASEC ACADEMY

IDOR - Santa's Little IDOR Report

WAPT Phase 1

Task 4 Common Web Vulnerabilities

Prepared by

Kinza Noor

Report date

3 October 2026

Learning platform

TryHackMe

Description:

This report documents an approach to testing object-level access controls in the TryPresentMe training application. It includes numeric, encoded and hashed references, with an optional voucher exercise. Observation fields and evidence placeholders must be completed from the actual lab session before submission.

Table of Contents

1 Objective 3
2 Concept and Understanding 3
2.1 Insecure Direct Object References 3
2.2 Authentication and Authorization 3
2.3 Horizontal and Vertical Escalation 3
2.4 Encoded Hashed and UUID References 3
2.5 Impact and Evidence 3
3 Practical Work 3
3.1 Environment and Login 3
3.2 Establishing the Account Baseline 3
3.3 Testing the Numeric Reference 4
3.4 Parent Account Challenge 4
3.5 Base64 and MD5 Child References 4
3.6 Optional Voucher Challenge 4
4 Tools Used 4
5 Evidence 5
6 Result 5
7 Key Learnings 6
8 References 6

1 Objective

To distinguish authentication from authorization, identify client-controlled object references and check whether the application restricts access to the correct user. The goal is to explain each observed access-control failure with a reproducible request and response.

2 Concept and Understanding

2.1 Insecure Direct Object References

IDOR stands for Insecure Direct Object Reference. It occurs when an application accepts an object identifier but fails to check the requester's permission for that object. The reference can appear in a path, query, body or another client-controlled field. Identifiers are necessary for many applications; missing authorization is the security weakness.

2.2 Authentication and Authorization

Authentication establishes identity, normally through credentials followed by a verified session. Authorization determines which resources and actions that identity may access. A user ID stored in the browser is editable and cannot be trusted as proof of identity. Protected data should be associated with the authenticated identity on the server.

2.3 Horizontal and Vertical Escalation

Horizontal escalation gives access to another user's records at a similar privilege level. Vertical escalation gives access to functions reserved for a higher role. Classification depends on the permission crossed. A changed dashboard label alone is insufficient evidence; inspect the server response for another user's protected information.

2.4 Encoded Hashed and UUID References

Base64 is reversible encoding. A hash of a small, predictable identifier can be reproduced for candidate values. Neither approach provides authorization. Complex random identifiers reduce guessing but still require access checks. UUID version 1 contains a timestamp, clock sequence and node value; a timestamp alone is insufficient to reconstruct a complete UUID.

2.5 Impact and Evidence

Unauthorized reads can affect confidentiality; unauthorized edits or voucher claims can affect integrity. Record only effects actually demonstrated. A successful HTTP status without another user's data does not establish an IDOR, and an unavailable object does not prove that access control is working.

3 Practical Work

3.1 Environment and Login

Start the room machine and use the AttackBox or VPN-connected Kali browser. The room supplies the following training credentials.

3.2 Establishing the Account Baseline

Open Developer Tools, select Network and refresh the dashboard. Inspect the account-information request and response. Record the exact URL, method, identifier and account details shown. Save the original response before changing anything.

3.3 Testing the Numeric Reference

In Local Storage, inspect the auth_user entry. The walkthrough changes user_id from 10 to 11, then refreshes the page. Preserve the login context and compare the resulting network response with the baseline. Restore the original entry afterward, or log out and sign in again.

3.4 Parent Account Challenge

Using the captured account request, test a small set of candidate IDs within the lab. Keep the session constant and record the returned parent and child count. Stop when the parent with ten children is identified. Burp Repeater may be used to compare individual requests; it is optional for this room.

Document the exact request that produced the answer. Do not use a guessed ID or a value copied from another learner as an observed finding.

3.5 Base64 and MD5 Child References

Restore the baseline account. Capture the requests generated by the child view and edit controls. Keep each original method, endpoint and required parameters, changing only the object reference. Inspect the response before attempting another value.

```
printf '%s' '2' | base64
printf '%s' '2' | md5sum
```

These commands encode or hash the text 2 without a trailing newline. Use the same input format as the captured application reference. For MD5, confirm the relationship using known examples; appearance or string length alone cannot establish the algorithm or input.

For the optional child challenge, look for the record born on 2019-04-17 and record its id_number. Compare responses rather than relying solely on status codes. Viewing an edit request does not require changing the child's stored data.

3.6 Optional Voucher Challenge

Inspect a voucher locally to identify its UUID version. Capture the request for /parents/vouchers/claim and record its method and input format before testing. The challenge concerns a voucher generated on a minute boundary between 20:00 and 24:00 UTC on 20 November 2025.

When investigating version 1 candidates, the node and clock-sequence fields also matter. Reconstructing candidates requires matching the lab's generation assumptions. A four-hour interval contains 240 minute boundaries if midnight is excluded; this is a candidate-time count, not a guarantee that any voucher will be valid.

4 Tools Used

Tool	Purpose
Browser Developer Tools	Inspect Network traffic and Local Storage.
Burp Repeater if used	Compare captured requests with changed references.
Base64 and md5sum	Reproduce candidate identifier transformations.
Local UUID inspection if used	Examine voucher fields and generation assumptions.
TryHackMe training machine	Provide the assigned TryPresentMe application.

5 Evidence

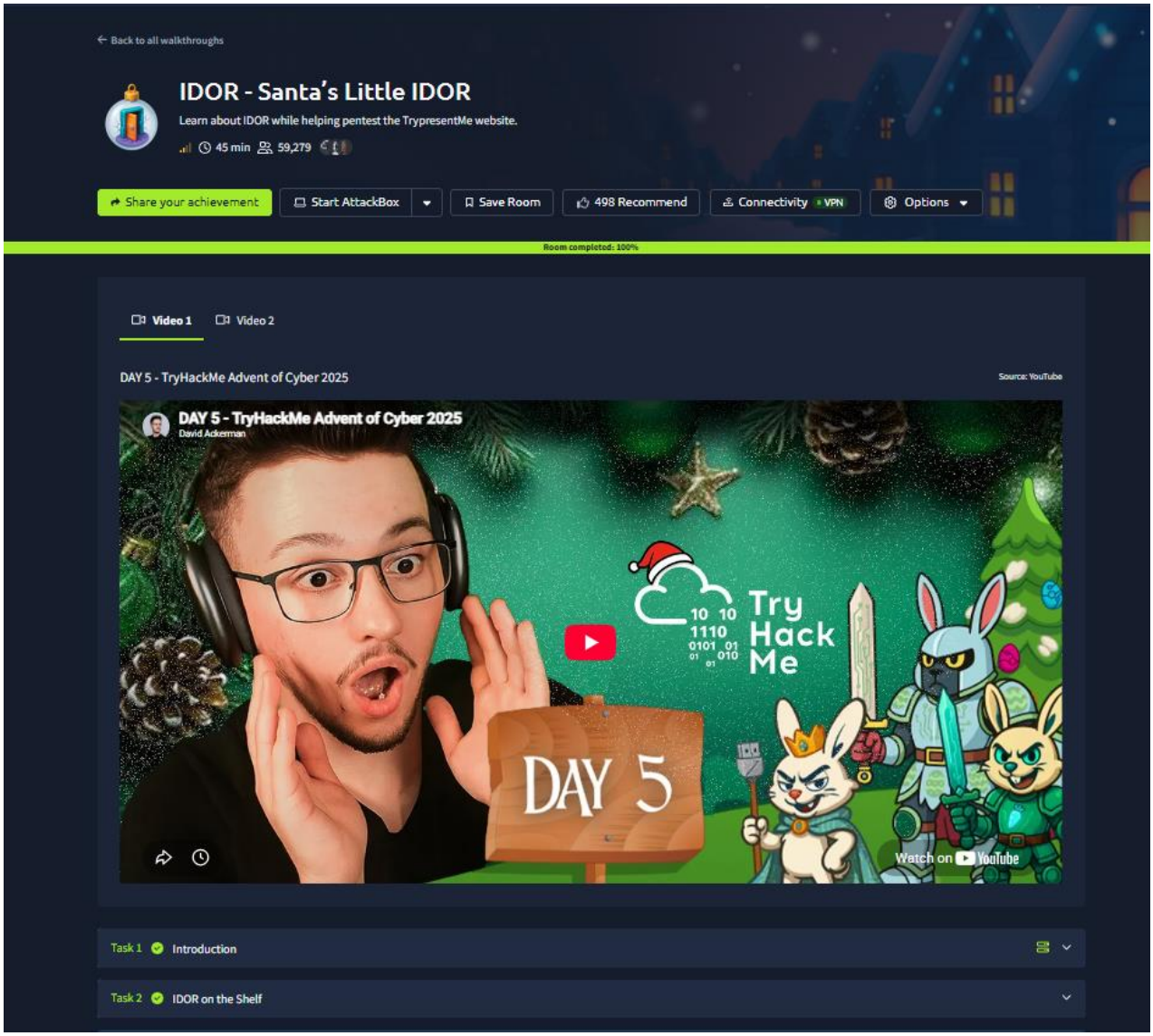
Insert screenshots that pair the changed reference with the matching response. Retain the relevant endpoint and lab data, and redact session tokens or unrelated personal information.

Figure	Screenshot to insert
1	Baseline account request and corresponding response.
2	Modified user_id and another account's server response.
3	Verified parent ID and the count of ten children.
4	Base64 or MD5 reference and matching child record, if completed.
5	Voucher request and successful lab result, if completed.
6	Room title and current completion progress.



6 Result

For each finding, state the authenticated user, target object, changed reference and returned data. Explain why that user lacked permission. Add the figure supporting the conclusion; distinguish room examples from your own observed results.



7 Key Learnings

Access to another user's record must be prevented on the server for every read and write operation. Scope object lookups to the authenticated user's permitted records and deny unauthorized requests consistently. Base64, MD5 and unpredictable IDs do not replace ownership checks. Retest with accounts having different permissions and monitor denied access attempts.

A secure object reference is an identifier backed by enforced permissions. For vouchers, also validate eligibility, expiry and claim state. Describe these as recommendations unless a fix has actually been implemented and verified in the lab.

8 References

TryHackMe. IDOR Santa's Little IDOR. https://tryhackme.com/room/idor-aoc2025-zl6MywQid9

OWASP. Insecure Direct Object Reference Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

IETF. RFC 9562 Universally Unique IDentifiers. https://www.rfc-editor.org/rfc/rfc9562.html